

UNIVERSIDADE FEDERAL DE SANTA MARIA
Curso de Engenharia de Computação / Redes de Computadores

Avaliação Parcial 2
Projeto de Redes de Computadores
Implementação do Protocolo SCHC
Caso de uso: Capítulo 8 - Ping Fragmentation

Aluno: Angelo Luigi Bocchi Lovatto

Disciplina: Redes de Computadores

Tema: SCHC, compressão e fragmentação em redes LPWAN

Ambiente: OpenSCHC VM, CORE Network Emulator e scripts do diretório MOOC-5

Sumário

1	Introdução	2
2	Justificativa da escolha do capítulo	2
3	Fundamentação teórica	3
3.1	SCHC	3
3.2	Modo NoAck	3
3.3	Direções DW e UP	3
4	Ambiente utilizado	4
5	Procedimento experimental	4
5.1	Inicialização do dispositivo e do núcleo	4
5.2	Teste 1: ping básico sem forçar fragmentação	5
5.3	Teste 2: ping grande sem regra de fragmentação	6
5.4	Adição da regra NoAck no sentido DW	7
5.5	Adição da regra NoAck no sentido UP	7
6	Arquivo final de regras	7
7	Resultado final	9
8	Análise dos resultados	10
9	Conclusão	11
	Referências	12
A	Checklist de execução	13
B	Comandos principais utilizados	13

1 Introdução

Este relatório apresenta a implementação de um caso de uso do protocolo SCHC (Static Context Header Compression and Fragmentation) no contexto da disciplina de Redes de Computadores. O trabalho foi desenvolvido com base nos padrões de Internet indicados no enunciado, principalmente a RFC 8724, que especifica o mecanismo SCHC, e a RFC 9011, que trata do uso do SCHC sobre LoRaWAN.

O SCHC é utilizado em cenários de redes LPWAN, nos quais os dispositivos possuem baixa capacidade de transmissão, baixo consumo de energia e enlaces com tamanho de carga útil limitado. Nesses ambientes, transmitir cabeçalhos IPv6, UDP ou ICMPv6 completos pode consumir uma parte significativa do quadro disponível. Por isso, o SCHC trabalha com dois mecanismos principais: a compressão de cabeçalhos por meio de regras compartilhadas e a fragmentação de pacotes quando a mensagem comprimida ainda é maior que a unidade máxima de transmissão do enlace.

Entre as opções propostas no trabalho, foram avaliados os capítulos 6, 7 e 8 do material *The Book of SCHC*. O capítulo escolhido foi o Capítulo 8, chamado *Ping fragmentation*, por apresentar um caso de uso direto, verificável e consistente: a geração de mensagens ICMPv6 maiores para forçar a fragmentação SCHC. O próprio material informa que esse capítulo continua o estudo da compressão de mensagens de ping, agora aumentando o tamanho das mensagens para que elas precisem ser fragmentadas, utilizando o diretório MOOC-5 como base de execução.

2 Justificativa da escolha do capítulo

A escolha do Capítulo 8 foi feita por ser o caminho mais adequado para entregar uma implementação completa e com resultados observáveis. O Capítulo 6, *Simple CoAP*, também é interessante, mas envolve servidor CoAP, cliente CoAP, uso de CBOR, opções CoAP e tratamento de erros. O Capítulo 7, *Fragmentation*, é mais amplo e conceitual, abordando vários modos de fragmentação, como NoAck, Ack Always e Ack on Error. Já o Capítulo 8 permite executar um experimento completo com três entidades: aplicação, núcleo SCHC e dispositivo.

No caso implementado, a aplicação envia mensagens ICMPv6 Echo Request para o dispositivo. Inicialmente, um ping normal é transmitido com sucesso. Em seguida, o tamanho do ping é aumentado com a opção `-s 1000`. Como o pacote comprimido passa a ser grande demais para ser enviado sem fragmentação, ocorre a falha esperada: o núcleo informa que não encontrou regra de fragmentação. Depois disso, são adicionadas regras SCHC de fragmentação no modo NoAck para os dois sentidos de comunicação.

3 Fundamentação teórica

3.1 SCHC

O SCHC combina compressão de cabeçalhos e fragmentação. A compressão depende de regras previamente conhecidas pelas entidades envolvidas. Essas regras descrevem campos dos protocolos, como versão IPv6, limite de saltos, tipo ICMPv6, identificador, número de sequência e carga útil. Quando os valores são conhecidos ou podem ser calculados, eles não precisam ser enviados integralmente no pacote comprimido.

A fragmentação é necessária quando, mesmo depois da compressão, o pacote continua maior que o limite de transmissão do meio. Em redes LPWAN, esse limite pode ser bastante restrito. Portanto, a fragmentação SCHC permite dividir a mensagem em fragmentos menores, enviá-los separadamente e reconstruí-los no destino.

3.2 Modo NoAck

O modo NoAck, utilizado neste trabalho, é um modo de fragmentação sem confirmação positiva a cada fragmento. O emissor divide a mensagem em fragmentos e envia todos em sequência. Os fragmentos intermediários são marcados como *All-0*. O último fragmento é marcado como *All-1* e carrega informações que permitem ao receptor verificar se a remontagem ocorreu corretamente.

Esse modo é simples e reduz o tráfego de retorno, característica útil em redes LPWAN, onde o canal de downlink pode ser escasso. Por outro lado, se um fragmento for perdido, não existe retransmissão automática baseada em confirmação, diferentemente de modos como Ack Always ou Ack on Error.

3.3 Direções DW e UP

Neste relatório foram usadas duas direções de fragmentação:

- **DW**: sentido de descida, da aplicação para o dispositivo, passando pelo núcleo SCHC.
- **UP**: sentido de subida, do dispositivo para a aplicação, também passando pelo núcleo SCHC.

A primeira regra adicionada resolveu apenas a ida da mensagem ICMPv6 Echo Request. Entretanto, o ping grande só passa a funcionar completamente quando também existe uma regra para o retorno, pois a resposta ICMPv6 Echo Reply possui tamanho semelhante ao pacote enviado.

4 Ambiente utilizado

O experimento foi realizado na máquina virtual OpenSCHC VM, baseada em Debian, contendo o ambiente de simulação CORE e os arquivos do projeto OpenSCHC. A topologia utilizada foi a topologia do cenário `schc-ping.xml`, composta por três nós:

- **iot_device**: representa o dispositivo restrito.
- **schc_core**: representa o núcleo responsável pelo tratamento SCHC.
- **app_server**: representa a aplicação que envia o ping.

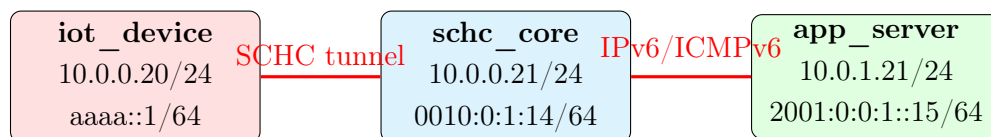


Figura 1: Representação esquemática da topologia usada no CORE. Esta figura é um diagrama explicativo; os prints reais da execução aparecem nas figuras de resultados.

Após iniciar a simulação no CORE, foram abertos três terminais, um em cada nó. Em todos os terminais foi acessado o diretório do Capítulo 8:

```
cd M00C-5
ls
```

A listagem confirmou a presença dos arquivos necessários:

```
icmp-bi.json ping_core1.py ping_dev1.py
```

As figuras de execução presentes neste relatório foram substituídas pelos prints reais coletados durante o experimento: o ping básico, a falha esperada sem fragmentação e o ping grande funcionando após a inclusão das regras DW e UP.

5 Procedimento experimental

5.1 Inicialização do dispositivo e do núcleo

No nó **device**, foi executado o script responsável por representar o dispositivo SCHC:

```
python3 ping_dev1.py
```

No nó **core**, foi executado o script responsável pelo núcleo SCHC:

```
python3 ping_core1.py
```

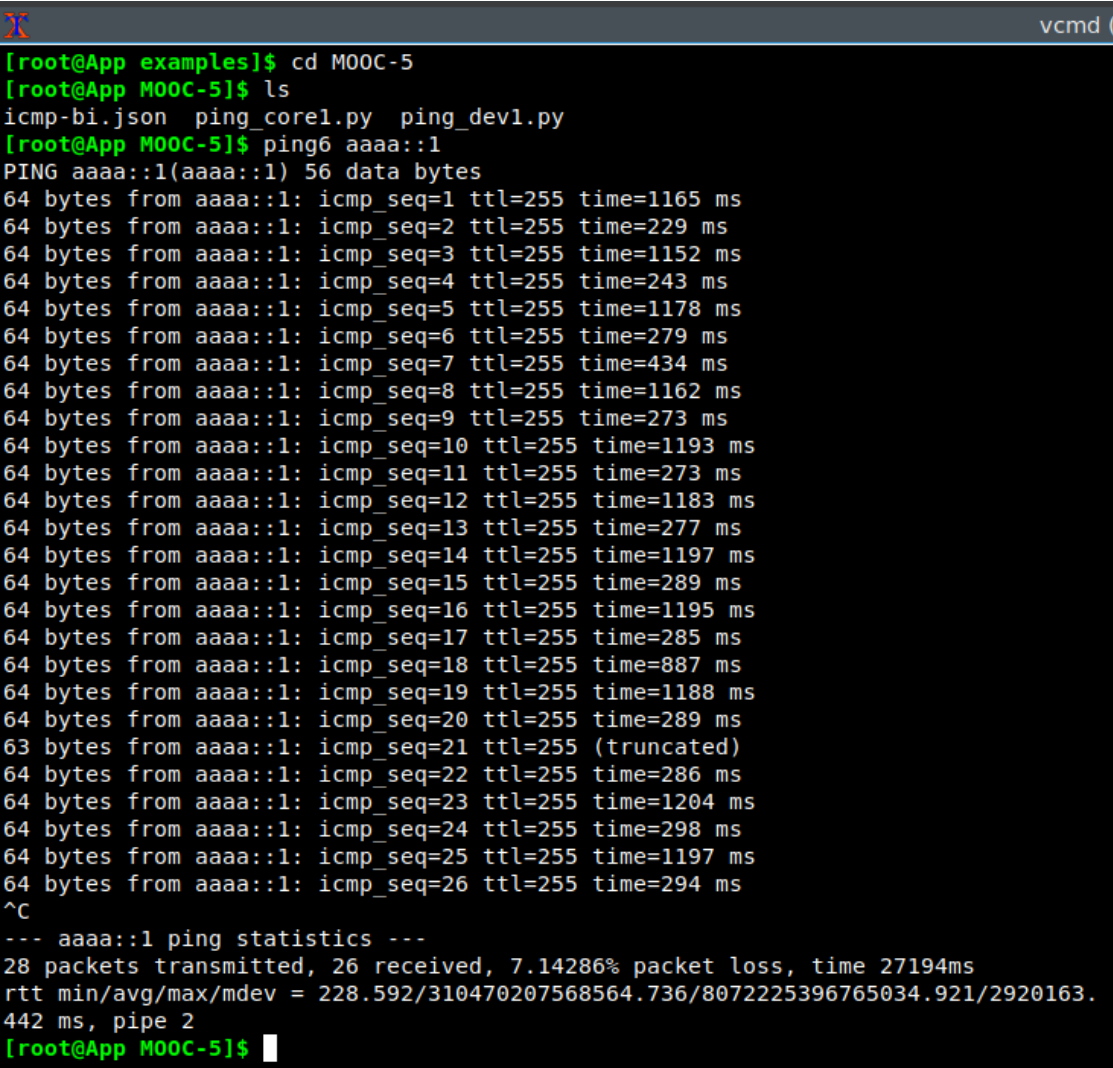
Com os dois scripts em execução, o nó App foi usado para enviar mensagens ICMPv6 para o endereço do dispositivo.

5.2 Teste 1: ping básico sem forçar fragmentação

O primeiro teste teve o objetivo de validar a comunicação básica entre aplicação, núcleo e dispositivo. Para isso, foi executado no nó App:

```
ping6 aaaa::1
```

O resultado observado indicou que o ping funcionou. No teste realizado, foram enviados 28 pacotes e 26 foram recebidos, com perda de aproximadamente 7,14%. Apesar da pequena perda, o objetivo desta etapa foi atingido, pois ficou demonstrado que a topologia estava ativa e que o caminho entre aplicação, núcleo SCHC e dispositivo estava operacional.



```
[root@App examples]$ cd M00C-5
[root@App M00C-5]$ ls
icmp-bi.json ping_core1.py ping_dev1.py
[root@App M00C-5]$ ping6 aaaa::1
PING aaaa::1(aaaa::1) 56 data bytes
64 bytes from aaaa::1: icmp_seq=1 ttl=255 time=1165 ms
64 bytes from aaaa::1: icmp_seq=2 ttl=255 time=229 ms
64 bytes from aaaa::1: icmp_seq=3 ttl=255 time=1152 ms
64 bytes from aaaa::1: icmp_seq=4 ttl=255 time=243 ms
64 bytes from aaaa::1: icmp_seq=5 ttl=255 time=1178 ms
64 bytes from aaaa::1: icmp_seq=6 ttl=255 time=279 ms
64 bytes from aaaa::1: icmp_seq=7 ttl=255 time=434 ms
64 bytes from aaaa::1: icmp_seq=8 ttl=255 time=1162 ms
64 bytes from aaaa::1: icmp_seq=9 ttl=255 time=273 ms
64 bytes from aaaa::1: icmp_seq=10 ttl=255 time=1193 ms
64 bytes from aaaa::1: icmp_seq=11 ttl=255 time=273 ms
64 bytes from aaaa::1: icmp_seq=12 ttl=255 time=1183 ms
64 bytes from aaaa::1: icmp_seq=13 ttl=255 time=277 ms
64 bytes from aaaa::1: icmp_seq=14 ttl=255 time=1197 ms
64 bytes from aaaa::1: icmp_seq=15 ttl=255 time=289 ms
64 bytes from aaaa::1: icmp_seq=16 ttl=255 time=1195 ms
64 bytes from aaaa::1: icmp_seq=17 ttl=255 time=285 ms
64 bytes from aaaa::1: icmp_seq=18 ttl=255 time=887 ms
64 bytes from aaaa::1: icmp_seq=19 ttl=255 time=1188 ms
64 bytes from aaaa::1: icmp_seq=20 ttl=255 time=289 ms
63 bytes from aaaa::1: icmp_seq=21 ttl=255 (truncated)
64 bytes from aaaa::1: icmp_seq=22 ttl=255 time=286 ms
64 bytes from aaaa::1: icmp_seq=23 ttl=255 time=1204 ms
64 bytes from aaaa::1: icmp_seq=24 ttl=255 time=298 ms
64 bytes from aaaa::1: icmp_seq=25 ttl=255 time=1197 ms
64 bytes from aaaa::1: icmp_seq=26 ttl=255 time=294 ms
^C
--- aaaa::1 ping statistics ---
28 packets transmitted, 26 received, 7.14286% packet loss, time 27194ms
rtt min/avg/max/mdev = 228.592/310470207568564.736/8072225396765034.921/2920163.
442 ms, pipe 2
[root@App M00C-5]$
```

Figura 2: Teste inicial com ping6 aaaa::1 funcionando no terminal do nó App.

Trecho do resultado observado:

```
[root@App MOOC-5]$ ping6 aaaa::1
PING aaaa::1(aaaa::1) 56 data bytes
64 bytes from aaaa::1: icmp_seq=1 ttl=255 time=1165 ms
64 bytes from aaaa::1: icmp_seq=2 ttl=255 time=229 ms
64 bytes from aaaa::1: icmp_seq=3 ttl=255 time=1152 ms
...
--- aaaa::1 ping statistics ---
28 packets transmitted, 26 received, 7.14286% packet loss
```

5.3 Teste 2: ping grande sem regra de fragmentação

Depois de validar o ping normal, foi executado um ping maior, com 1000 bytes de dados, enviando apenas um pacote:

```
ping6 -s 1000 -c 1 aaaa::1
```

Nessa etapa, o comportamento esperado era a falha, pois o pacote comprimido ficou maior que o limite aceito para envio sem fragmentação. O terminal do App indicou perda total:

```
[root@App MOOC-5]$ ping6 -s 1000 -c 1 aaaa::1
PING aaaa::1(aaaa::1) 1000 data bytes

--- aaaa::1 ping statistics ---
1 packets transmitted, 0 received, 100% packet loss, time 0ms
```

No terminal do core, apareceu a mensagem que confirma a causa da falha:

```
make_frag_session, device_id: udp:10.0.0.20:8888 core_id None direction DW
Protocol fragmentation rule not found
```

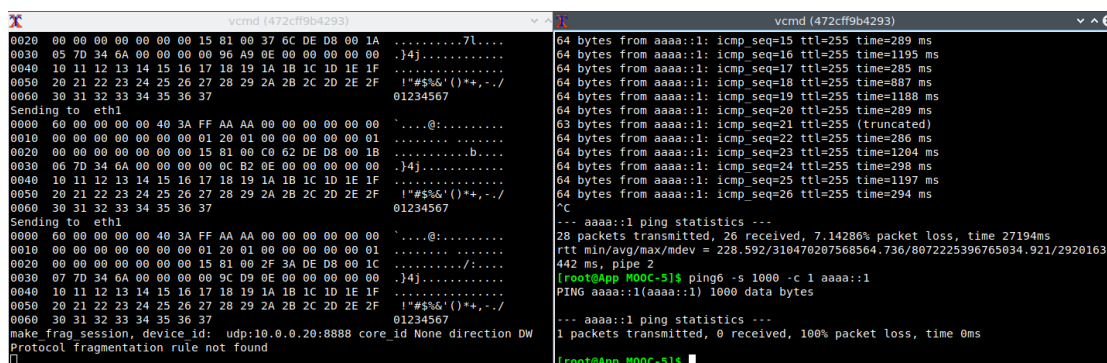


Figura 3: Falha esperada do ping grande antes da criação da regra de fragmentação SCHC.

Esse resultado mostra que a compressão foi realizada, mas o sistema não possuía uma regra de fragmentação associada à direção DW. Portanto, o núcleo SCHC não sabia como dividir o pacote em fragmentos menores.

5.4 Adição da regra NoAck no sentido DW

Para resolver a fragmentação no caminho da aplicação para o dispositivo, foi adicionada a regra de fragmentação NoAck no arquivo `icmp-bi.json`. Essa regra usa RuleID 10 com comprimento de 5 bits e direção DW:

```
1 {  
2   "RuleID": 10,  
3   "RuleIDLength": 5,  
4   "Fragmentation": {  
5     "FRMode": "NoAck",  
6     "FRDirection": "DW"  
7   }  
8 }
```

Após essa alteração, o pacote grande passou a ser fragmentado no sentido App para Device. Entretanto, a aplicação ainda não recebeu a resposta, porque a regra adicionada só resolvia o caminho de ida. O retorno do Echo Reply também precisa ser fragmentado.

5.5 Adição da regra NoAck no sentido UP

Para concluir o funcionamento do ping grande, foi adicionada uma segunda regra de fragmentação no mesmo arquivo `icmp-bi.json`, agora com direção UP:

```
1 {  
2   "RuleID": 11,  
3   "RuleIDLength": 5,  
4   "Fragmentation": {  
5     "FRMode": "NoAck",  
6     "FRDirection": "UP"  
7   }  
8 }
```

Com isso, o arquivo passou a conter uma regra para o envio do pacote grande e outra para o retorno da resposta.

O código completo do arquivo final de regras é apresentado na Seção seguinte, em formato textual, para evitar o uso de prints ilustrativos ou não reais.

6 Arquivo final de regras

O arquivo final utilizado no experimento ficou da seguinte forma:

```

1 {
2   "DeviceID": "udp:10.0.0.20:8888",
3   "SoR": [
4     {
5       "RuleID": 6,
6       "RuleIDLength": 3,
7       "Compression": [
8         {"FID": "IPV6.VER", "TV": 6, "MO": "equal", "CDA": "not-sent"},
9         {"FID": "IPV6.TC", "TV": 0, "MO": "equal", "CDA": "not-sent"},
10        {"FID": "IPV6.FL", "TV": 0, "MO": "ignore", "CDA": "not-sent"},
11        {"FID": "IPV6.LEN", "MO": "ignore", "CDA": "compute-length"},
12        {"FID": "IPV6.NXT", "TV": 58, "MO": "equal", "CDA": "not-sent"},
13        {"FID": "IPV6.HOP_LMT", "TV": 255, "MO": "ignore", "CDA": "not-sent"},
14        {"FID": "IPV6.DEV_PREFIX", "TV": "AAAA::/64", "MO": "equal", "CDA": "not-sent"},
15        {"FID": "IPV6.DEV_IID", "TV": "::1", "MO": "equal", "CDA": "not-sent"},
16        {"FID": "IPV6.APP_PREFIX", "MO": "ignore", "CDA": "value-sent"},
17        {"FID": "IPV6.APP_IID", "MO": "ignore", "CDA": "value-sent"},
18        {"FID": "ICMPV6.TYPE", "TV": 128, "DI": "DW", "MO": "equal", "CDA": "not-sent"},
19        {"FID": "ICMPV6.TYPE", "TV": 129, "DI": "UP", "MO": "equal", "CDA": "not-sent"},
20        {"FID": "ICMPV6.CODE", "TV": 0, "MO": "equal", "CDA": "not-sent"},
21        {"FID": "ICMPV6.CKSUM", "TV": 0, "MO": "ignore", "CDA": "compute-checksum"},
22        {"FID": "ICMPV6.IDENT", "TV": 0, "MO": "ignore", "CDA": "value-sent"},
23        {"FID": "ICMPV6.SEQNO", "TV": 0, "MO": "ignore", "CDA": "value-sent"},
24        {"FID": "ICMPV6.PAYLOAD", "TV": 0, "MO": "ignore", "CDA": "value-sent"}
25      ]
26    },
27    {
28      "RuleID": 10,
29      "RuleIDLength": 5,
30      "Fragmentation": {
31        "FRMode": "NoAck",
32        "FRDirection": "DW"
33      }
34    },
35    {
36      "RuleID": 11,
37      "RuleIDLength": 5,
38      "Fragmentation": {
39        "FRMode": "NoAck",
40        "FRDirection": "UP"
41      }
42    }
43  ]
44 }

```

7 Resultado final

Após a inclusão das regras de fragmentação nos dois sentidos, os scripts foram executados novamente:

```
# No device
python3 ping_dev1.py

# No core
python3 ping_core1.py

# No App
ping6 -s 1000 -c 1 aaaa::1
```

O resultado final no nó App foi:

```
[root@App M00C-5]$ ping6 -s 1000 -c 1 aaaa::1
PING aaaa::1(aaaa::1) 1000 data bytes
1008 bytes from aaaa::1: icmp_seq=1 ttl=255 time=7478 ms

--- aaaa::1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 7478.467/7478.467/7478.467/0.000 ms
```

```
[root@App M00C-5]$ ping6 -s 1000 -c 1 aaaa::1
PING aaaa::1(aaaa::1) 1000 data bytes
1008 bytes from aaaa::1: icmp_seq=1 ttl=255 time=7478 ms

--- aaaa::1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 7478.467/7478.467/7478.467/0.000 ms
[root@App M00C-5]$
```

Figura 4: Ping grande funcionando após a inclusão das regras NoAck DW e UP.

Esse resultado confirma que a fragmentação SCHC foi implementada com sucesso. O pacote ICMPv6 com 1000 bytes de dados foi enviado, fragmentado, remontado e respondido. A estatística final mostra um pacote transmitido, um pacote recebido e 0% de perda.

A evidência gráfica final do funcionamento está no teste do nó App, no qual o pacote de 1000 bytes foi transmitido e respondido com sucesso. O arquivo de regras apresentado anteriormente documenta as duas regras de fragmentação utilizadas no experimento.

8 Análise dos resultados

A Tabela 1 resume os principais testes executados.

Tabela 1: Resumo dos testes realizados

Teste	Comando	Resultado observado
Ping básico	<code>ping6 aaaa::1</code>	Comunicação funcional entre App, Core e Device. Foram enviados 28 pacotes e 26 recebidos.
Ping grande sem regra	<code>ping6 -s 1000 -c 1 aaaa::1</code>	Falha esperada. O App teve 100% de perda e o Core exibiu <code>Protocol fragmentation rule not found</code> .
Ping grande com DW	<code>ping6 -s 1000 -c 1 aaaa::1</code>	A ida passou a ser fragmentada, mas a resposta ainda não retornou ao App por falta da regra UP.
Ping grande com DW e UP	<code>ping6 -s 1000 -c 1 aaaa::1</code>	Sucesso. O App recebeu <code>1008 bytes from aaaa::1</code> , com 1 pacote transmitido, 1 recebido e 0% de perda.

O comportamento observado está de acordo com o objetivo do capítulo escolhido. O erro inicial não representa uma falha do ambiente, mas sim uma etapa necessária para demonstrar que o pacote grande exigia uma regra de fragmentação. Ao adicionar a regra `RuleID 10/5` no sentido DW, o sistema passou a fragmentar o pacote da aplicação para o dispositivo. Porém, como o Echo Reply também era grande, foi necessário adicionar a regra `RuleID 11/5` no sentido UP. Somente com as duas regras o ciclo completo do ping foi concluído.

O tempo de resposta final, cerca de 7478 ms, foi elevado. Isso é aceitável no contexto do experimento, pois o teste foi realizado em uma máquina virtual com emulação de rede e processamento SCHC em software. Para o objetivo do trabalho, o dado principal é a entrega bem-sucedida do pacote grande, indicada pelo recebimento de 1 pacote em 1 transmitido.

9 Conclusão

O trabalho implementou com sucesso o caso de uso do Capítulo 8, *Ping fragmentation*, do material *The Book of SCHC*. A implementação foi realizada na OpenSCHC VM, usando o emulador CORE e os arquivos do diretório MOOC-5.

Inicialmente, foi validada a comunicação básica entre aplicação, núcleo e dispositivo por meio de um ping IPv6 comum. Em seguida, foi executado um ping maior, com 1000 bytes, que falhou corretamente por ausência de regra de fragmentação. Essa falha foi importante para comprovar que a mensagem comprimida precisava ser dividida antes de ser transmitida.

A solução consistiu em adicionar duas regras de fragmentação NoAck no arquivo `icmp-bi.json`: uma para o sentido DW, com `RuleID 10`, e outra para o sentido UP, com `RuleID 11`. Após a inclusão dessas regras, o ping grande passou a funcionar, retornando 0% de perda no teste final.

Assim, o experimento demonstrou na prática o funcionamento do SCHC como mecanismo de compressão e fragmentação para redes com restrição de tamanho de quadro. A implementação também mostrou a importância de configurar regras para os dois sentidos da comunicação quando tanto a requisição quanto a resposta podem exceder o limite de transmissão.

Referências

Referências

- [1] MINABURO, A.; TOUTAIN, L.; GOMEZ, C.; BARTHEL, D.; ZUNIGA, J. C. **RFC 8724: SCHC: Generic Framework for Static Context Header Compression and Fragmentation**. IETF, 2020. Disponível em: <https://www.rfc-editor.org/rfc/rfc8724>.
- [2] MINABURO, A.; TOUTAIN, L. **RFC 9011: Static Context Header Compression and Fragmentation over LoRaWAN**. IETF, 2021. Disponível em: <https://www.rfc-editor.org/rfc/rfc9011>.
- [3] TOUTAIN, L.; PELOV, A.; MINABURO, A. **The Book of SCHC**. Material de apoio utilizado no experimento, Capítulo 8 - Ping Fragmentation.
- [4] OpenSCHC. **OpenSCHC Virtual Machine**. Disponível em: <https://vm.openschc.net/>.
- [5] OpenSCHC. **OpenSCHC GitHub Repository**. Disponível em: <https://github.com/openschc/openschc>.

A Checklist de execução

1. Abrir a OpenSCHC VM no VirtualBox.
2. Abrir o CORE e carregar o cenário `schc-ping.xml`.
3. Iniciar a simulação com o botão *Play*.
4. Abrir terminal nos nós `device`, `core` e `App`.
5. Acessar o diretório `M00C-5` nos três nós.
6. Executar `python3 ping_dev1.py` no `device`.
7. Executar `python3 ping_core1.py` no `core`.
8. Testar `ping6 aaaa::1` no `App`.
9. Testar `ping6 -s 1000 -c 1 aaaa::1` sem regra e observar a falha.
10. Editar `icmp-bi.json`, adicionando regras `NoAck DW` e `UP`.
11. Reexecutar os scripts e testar novamente o ping grande.
12. Confirmar o resultado final com `1 packets transmitted, 1 received, 0% packet loss`.

B Comandos principais utilizados

```
# Entrar no diretorio do experimento
cd M00C-5
ls

# Executar no device
python3 ping_dev1.py

# Executar no core
python3 ping_core1.py

# Teste basico no App
ping6 aaaa::1

# Teste grande no App
ping6 -s 1000 -c 1 aaaa::1

# Validar o arquivo JSON depois da edicao
python3 -m json.tool icmp-bi.json > /dev/null && echo "JSON OK"
```